

**Universitatea Tehnica Gheorghe Asachi din Iasi**  
**Facultatea de Automatică și Calculatoare**  
**Grupa: 1309B**

# **PyBro**

## **Python IDE — Documentație de Proiect**

**Autori:**  
Filip Andrei-Robert  
Beligan Tudor  
Pînzaru Alexandru-Gabriel  
Modreanu Ștefan

Iași, 2026

## Cuprins

1. Introducere
2. Documentul Specificărilor Cerințelor (SRS)
  - 2.1. Scopul documentului
  - 2.2. Descrierea generală a produsului
  - 2.3. Cerințe funcționale
  - 2.4. Cerințe nonfuncționale
  - 2.5. Constrângeri
3. Șabloane de proiectare utilizate
4. Diagrame UML
  - 4.1. Diagrama de clase
  - 4.2. Diagrama cazurilor de utilizare
  - 4.3. Diagrama de activități
  - 4.4. Diagrama de secvențe
5. Modul de utilizare a programului
6. Tratarea excepțiilor
7. Structura modulară (DLL-uri)
8. Respectarea standardelor de codare
9. Capturi de ecran
- Anexă: Listing cod sursă

## 1. Introducere

PyBro este un mediu de dezvoltare integrat (IDE) destinat scrierii și executării codului Python, implementat în limbajul C# cu ajutorul bibliotecii IronPython. Proiectul a fost realizat în cadrul disciplinei Ingineria Programării de o echipă de patru studenți și respectă arhitectura MVC (Model-View-Controller).

Aplicația oferă utilizatorului posibilitatea de a edita fișiere sursă Python, de a le executa direct din interfață și de a vizualiza rezultatele în consola integrată. Printre funcționalitățile principale se numără: crearea, deschiderea și salvarea fișierelor Python, execuția scripturilor cu afișarea ieșirii standard și a erorilor, navigarea în arborele de directoare al proiectului, auto-indentare inteligentă, evidențierea sintaxei Python (syntax highlighting) prin intermediul bibliotecii AvalonEdit și un meniu de ajutor (Help) integrat.

Structura modulară a proiectului — organizată în 7 proiecte compilate ca DLL-uri independente — permite extinderea ușoară a funcționalităților și înlocuirea componentelor individuale fără a afecta restul sistemului.

## 2. Documentul Specificărilor Cerințelor (SRS)

### 2.1. Scopul documentului

Acest document descrie cerințele funcționale și nonfuncționale ale aplicației PyBro, un mediu de dezvoltare integrat pentru Python, implementat în C#. Documentul este structurat conform modelului IEEE pentru specificarea cerințelor software și este adresat atât echipei de dezvoltare, cât și evaluatorilor proiectului.

### 2.2. Descrierea generală a produsului

PyBro este o aplicație desktop dezvoltată în C# care integrează un interpretor Python (IronPython) pentru a permite scrierea, salvarea și execuția scripturilor Python. Interfața grafică este implementată folosind WPF (Windows Presentation Foundation), iar editorul de cod utilizează biblioteca AvalonEdit pentru syntax highlighting.

Componentele principale ale sistemului sunt:

- Modulul Model (PyBro.MVC) — gestionează logica aplicației, interpretorul Python și managementul fișierelor
- Modulul View (PyBro.MVC) — coordonează afișarea datelor prin intermediul adaptorului de UI
- Modulul Controller (PyBro.MVC) — coordonează comunicarea dintre Model și View prin ciclul Tick
- Modulul UI (PyBro.Ui) — interfața grafică WPF: MainWindow, HelpWindow, UiAdapter, TreeDir, FileItem
- Modulul Core (PyBro.Core) — clase de suport: ArrayListTextBuffer, FileManager, PythonInterpreter, DynamicLoader
- Modulul Contracts (PyBro.Contracts) — interfețe și contracte: IModel, IView, IController, IFileManager, IPythonInterpreter, ITextBuffer, IUiAdapter, ITreeDir, ICommand, IModelCommand, IViewCommand, MessageQueues, FileException, PythonException
- Modulul Commands (PyBro.Commands) — implementări ale comenzilor: ModelCommandRunPythonScript, ModelCommandSaveBuffer, ModelCommandLoadFile, ModelCommandRewriteBuffer, ViewCommandDisplayPythonError, ViewCommandRewriteActiveBuffer, ViewCommandSendOutputToConsole, ViewCommandUpdateTitle

### 2.3. Cerințe funcționale

Aplicația trebuie să îndeplinească următoarele cerințe funcționale:

- Crearea de fișiere Python noi
- Deschiderea și editarea fișierelor Python existente prin dialog sau din arborele de directoare
- Salvarea fișierelor editate pe disc
- Executarea scripturilor Python și afișarea rezultatelor (stdout și stderr) în consola integrată
- Auto-indentare inteligentă bazată pe analiza conținutului bufferului curent
- Evidențierea sintaxei Python (syntax highlighting) prin AvalonEdit
- Navigarea în arborele de directoare al proiectului și deschiderea fișierelor prin click
- Afișarea unui meniu de ajutor (Help) cu informații despre utilizarea aplicației
- Suport pentru operațiile de editare uzuale: Copy, Cut, Paste în editorul de cod

## **2.4. Cerințe nonfuncționale**

- Performanță: Execuția unui script Python simplu nu trebuie să blocheze GUI.
- Portabilitate: Aplicația rulează pe sisteme Windows cu .NET instalat
- Menținabilitate: Fiecare modul este implementat într-un DLL separat pentru a facilita actualizările independente
- Usabilitate: Interfața este intuitivă și accesibilă utilizatorilor fără cunoștințe de programare
- Fiabilitate: Excepțiile sunt tratate corespunzător și utilizatorul este notificat în caz de eroare
- Extensibilitate: Arhitectura bazată pe interfețe și șabloane de proiectare facilitează adăugarea de noi funcționalități

## **2.5. Constrângeri**

- Limbajul de implementare: C#
- Framework: .NET
- Interpretorul Python este furnizat de biblioteca IronPython
- Interfața grafică este implementată folosind WPF (Windows Presentation Foundation)
- Editorul de cod utilizează biblioteca AvalonEdit

### 3. Șabloane de proiectare utilizate

Proiectul PyBro utilizează trei șabloane de proiectare (design patterns), fiecare ales pentru a rezolva o problemă specifică de arhitectură:

#### 3.1. Builder (IdeBuilder)

Șablonul Builder este utilizat în clasa IdeBuilder din modulul PyBro.IDE pentru a construi obiectul principal Ide. Acesta permite configurarea pas cu pas a tuturor dependențelor (PythonInterpreter, FileManager, UiAdapter, TreeDir, MainWindow) înainte de crearea instanței finale, validând că toate componentele necesare au fost furnizate.

Exemplu de utilizare (Program.cs):

```
var ide = new IdeBuilder()  
    .WithApp(new System.Windows.Application())  
    .WithMainWindow(mainWindow)  
    .WithPythonInterpreter(new PythonInterpreter())  
    .WithFileManager(new FileManager())  
    .WithUiAdapter(new UiAdapter(mainWindow))  
    .WithTreeDir(new TreeDir())  
    .Build();
```

#### 3.2. Command (ICommand / IModelCommand / IViewCommand)

Șablonul Command este utilizat extensiv în modulele PyBro.Commands și Pybro.Contracts. Fiecare operație din aplicație (rulare script, salvare fișier, încărcare fișier, actualizare buffer, afișare eroare) este encapsulată într-un obiect comandă care implementează interfața ICommand cu metoda Execute(object[] args). Comenzile sunt transmise asincron prin MessageQueues (cozi concurente), separând producătorul de comandă de executantul acesteia.

Comenzi Model: ModelCommandRunPythonScript, ModelCommandSaveBuffer, ModelCommandLoadFile, ModelCommandRewriteBuffer.

Comenzi View: ViewCommandDisplayPythonError, ViewCommandRewriteActiveBuffer, ViewCommandSendOutputToConsole, ViewCommandUpdateTitle.

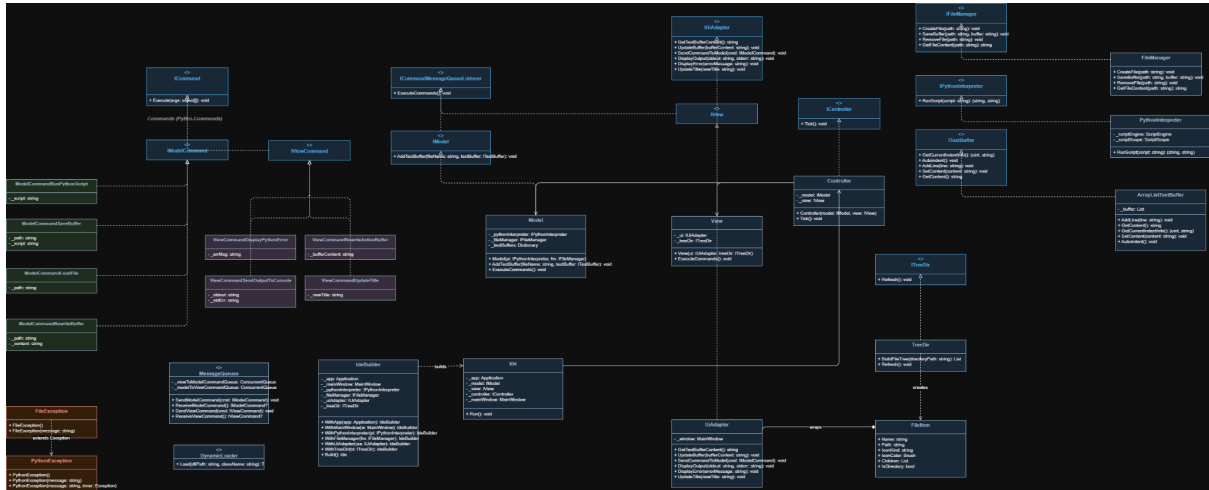
#### 3.3. Adapter (UiAdapter)

Șablonul Adapter este utilizat în clasa UiAdapter din modulul PyBro.Ui. Aceasta adaptează interfața specifică WPF a clasei MainWindow la contractul IUiAdapter definit în Pybro.Contracts, permițând ca View și comenzile de View să opereze independent de implementarea concretă a interfeței grafice.

## 4. Diagrame UML

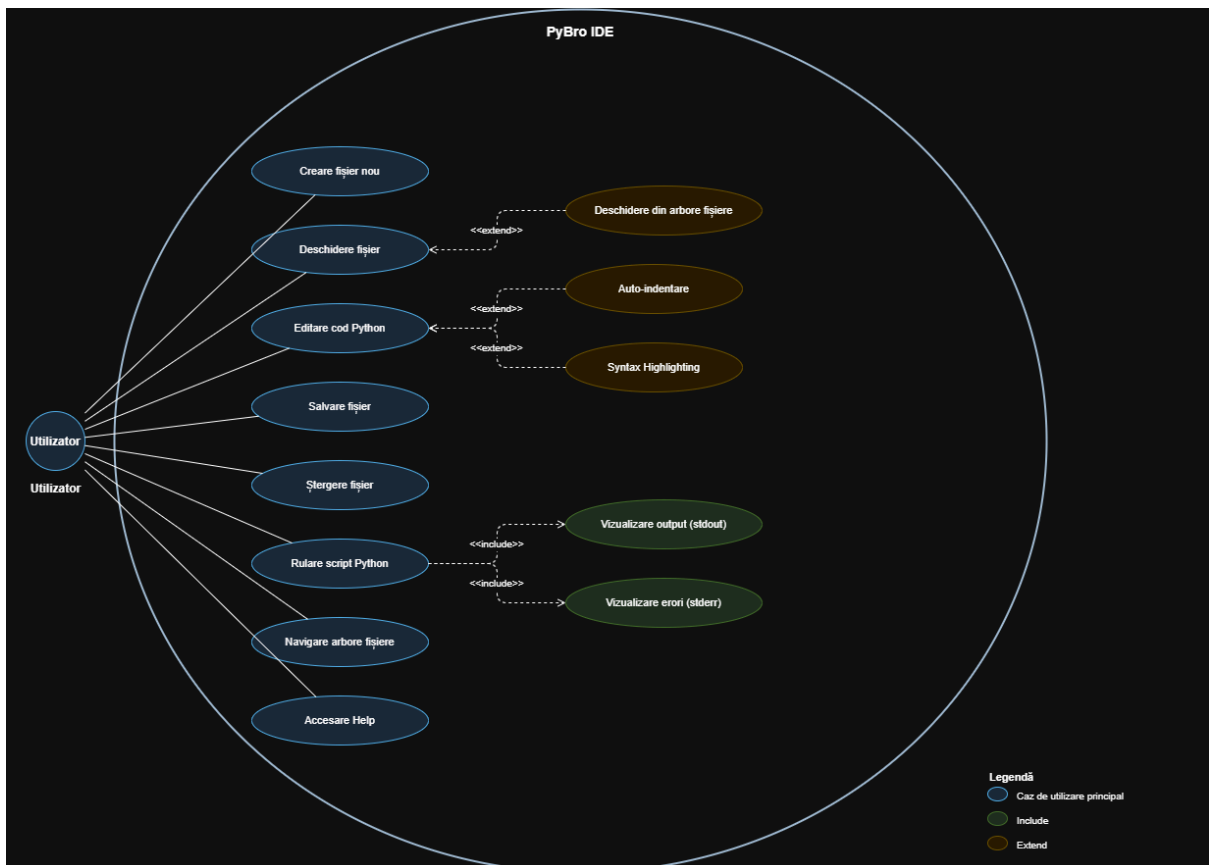
### 4.1. Diagrama de clase

Diagrama de clase prezintă structura internă a aplicației PyBro, incluzând toate clasele, interfețele și relațiile dintre acestea. Arhitectura respectă principiul MVC și utilizează interfețe pentru a decupla implementările concrete.



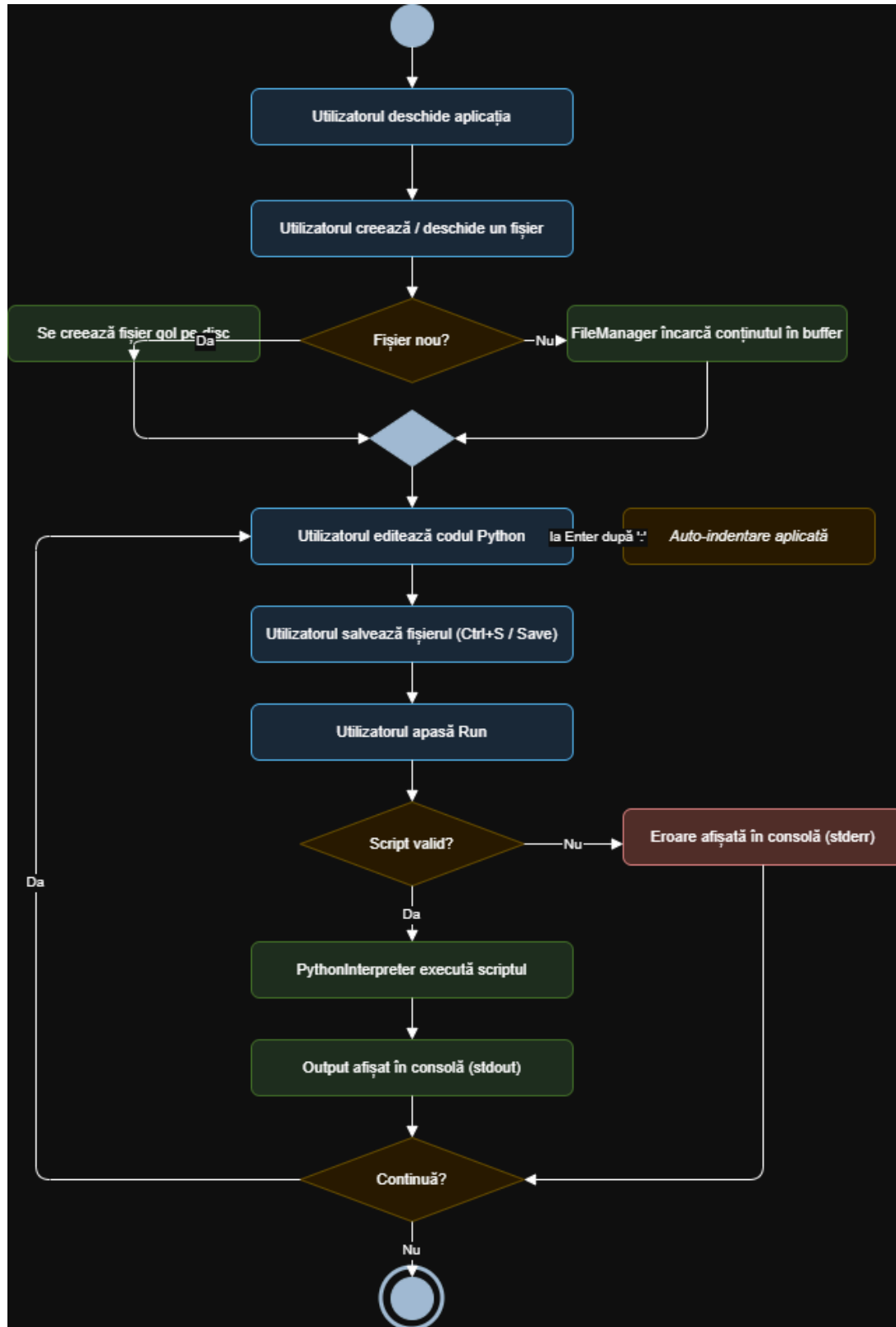
### 4.2. Diagrama cazurilor de utilizare

Diagrama cazurilor de utilizare prezintă interacțiunile posibile dintre utilizator și sistem.



### 4.3. Diagrama de activități

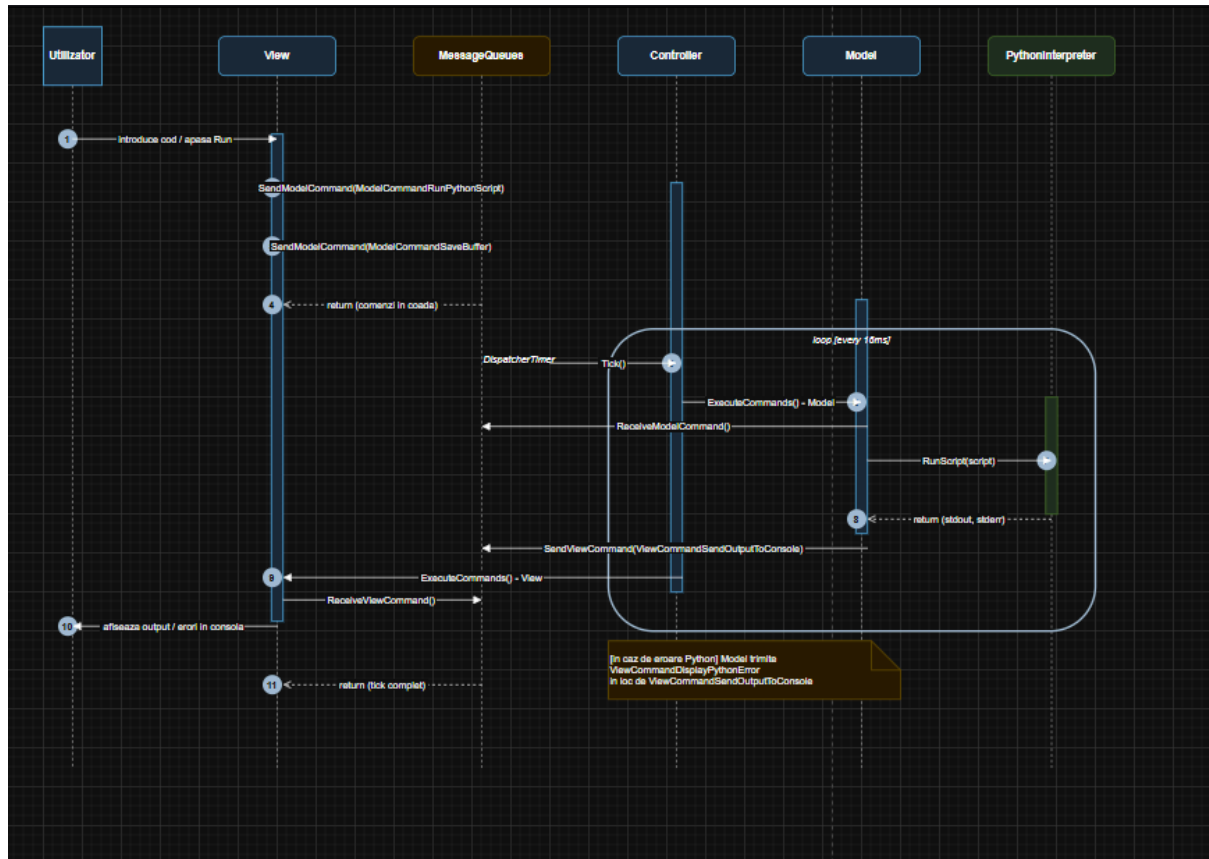
Diagrama de activități descrie fluxul de execuție al principalelor operații din aplicație, în special procesul de scriere și rulare a unui script Python.





#### 4.4. Diagrama de secvențe

Diagrama de secvențe ilustrează ordinea mesajelor schimbate între componentele sistemului în cadrul unui ciclu de execuție (Tick).



## 5. Modul de utilizare a programului

Această secțiune descrie modul de utilizare a aplicației PyBro din perspectiva unui utilizator care nu are cunoștințe de programare.

### 5.1. Lansarea aplicației

Aplicația se lansează prin dublu-click pe fișierul executabil PyBro.exe. La pornire, se deschide fereastra principală a aplicației, care conține editorul de cod (zona centrală), panoul de fișiere (stânga) și consola de output (jos).

### 5.2. Selectarea unui folder de lucru

Prin butonul dedicat din panoul lateral, utilizatorul poate selecta un folder de pe disc. Arborele de directoare se populează automat, afișând doar fișierele Python (.py) și directoarele.

### 5.3. Deschiderea unui fișier existent

Fișierele existente pot fi deschise fie prin butonul Load (care deschide un dialog de selectare a fișierelor), fie prin dublu-click pe fișierul dorit din arborele de directoare din panoul lateral. Conținutul fișierului apare automat în editorul de cod.

### 5.4. Editarea codului

Codul Python se editează direct în zona centrală a editorului. Aplicația oferă evidențierea sintaxei Python (cuvinte cheie, șiruri, comentarii sunt colorate diferit) și auto-indentare inteligentă la apăsarea tastei Enter după o linie care se termină cu ':' (de exemplu, after if, for, def, class).

### 5.5. Salvarea fișierului

Fișierul curent se salvează prin butonul Save. Dacă fișierul nu a fost salvat anterior, se deschide un dialog pentru alegerea numelui și locației. Salvările ulterioare suprascriu fișierul existent fără a solicita confirmarea locației.

### 5.6. Rularea unui script

Pentru a executa scriptul curent, utilizatorul apasă butonul Run. Dacă nu există un fișier curent, aplicația solicită salvarea înaintea execuției. Rezultatele execuției (output standard) sunt afișate în consola din partea inferioară a ferestrei. În caz de eroare în scriptul Python, mesajul de eroare este afișat tot în consolă, prefixat cu [ERROR].

### 5.7. Meniul Help

Meniul Help se accesează prin butonul Help din bara de meniu principală. Se deschide o fereastră separată (HelpWindow) care poate fi mutată independent și care oferă informații despre funcționalitățile aplicației.

## 6. Tratarea excepțiilor

Aplicația PyBro utilizează un sistem de excepții personalizate pentru a semnala erorile la nivel de operații de fișiere și de execuție Python. Sunt definite două clase de excepții în modulul Pybro.Contracts:

- `FileException` — pentru erori legate de operații pe sistemul de fișiere
- `PythonException` — pentru erori apărute în timpul execuției scripturilor Python

### 6.1. Tratarea erorilor de fișier (`FileManager.cs`)

Clasa `FileManager` tratează excepțiile sistemului de operare și le convertește în `FileException`, oferind mesaje clare utilizatorului:

```
public void CreateFile(string path)
{
    try
    {
        File.Create(path).Dispose();
    }
    catch
    {
        throw new FileException("Can not create file " + path);
    }
}

public void RemoveFile(string path)
{
    if (!File.Exists(path))
    {
        throw new FileException("File " + path + " does not exist!");
    }
    File.Delete(path);
}
```

### 6.2. Tratarea erorilor Python (`PythonInterpreter.cs`)

Clasa `PythonInterpreter` tratează separat excepțiile `IronPython` și excepțiile generale, asigurând că utilizatorul primește întotdeauna un mesaj de eroare relevant:

```
public (string, string) RunScript(string script)
{
    try
    {
        // ... executie script, redirect stdout/stderr ...
        _scriptEngine.Execute(script, _scriptScope);
        return (outText, errText);
    }
    catch (IronPython.Runtime.Exceptions.RuntimeException ironPythonEx)
    {
        throw new Exception("IronPython Runtime Exception: " +
            ironPythonEx.Message);
    }
    catch
    {
        throw new PythonException("Error while trying to run the python
script!");
    }
}
```

```
    }  
}
```

### 6.3. Propagarea erorilor către UI (ModelCommandRunPythonScript.cs)

Comenzile de model prind excepțiile și le transmit View-ului prin mecanismul de comenzi, fără a expune detalii tehnice utilizatorului:

```
public void Execute(object[] args)  
{  
    if (args[0] is IPythonInterpreter pi)  
    {  
        try  
        {  
            var (stdout, stderr) = pi.RunScript(_script);  
            var cmd = new ViewCommandSendOutputToConsole(stdout, stderr);  
            MessageQueues.SendViewCommand(cmd);  
        }  
        catch (PythonException pe)  
        {  
            var cmd = new ViewCommandDisplayPythonError(pe.Message);  
            MessageQueues.SendViewCommand(cmd);  
        }  
    }  
}
```

## 7. Structura modulară (DLL-uri)

Proiectul PyBro este organizat în 7 proiecte C# separate, fiecare compilat ca un DLL independent. Această structură asigură separarea clară a responsabilităților și permite înlocuirea sau actualizarea oricărui modul fără a recompila întregul proiect.

| Proiect         | Namespace          | Responsabilitate                                                    |
|-----------------|--------------------|---------------------------------------------------------------------|
| PyBro.Contracts | PyBro.Contracts    | Interfețe, excepții, MessageQueues — contractul comun al aplicației |
| PyBro.Core      | PyBro / PyBro.Core | FileManager, PythonInterpreter, ArrayListTextBuffer, DynamicLoader  |
| PyBro.Commands  | PyBro.Commands     | Toate clasele de comenzi Model și View                              |
| PyBro.MVC       | PyBro.MVC          | Model, View, Controller — implementarea arhitecturii MVC            |
| PyBro.Ui        | PyBro.UI           | MainWindow, HelpWindow, UiAdapter, TreeDir, FileItem — UI WPF       |
| PyBro.IDE       | PyBro.IDE          | IdeBuilder, Ide, Program — punctul de intrare al aplicației         |
| PyBro.Tests     | PyBro.Tests        | 21 teste unitare XUnit pentru clasa FileManager                     |

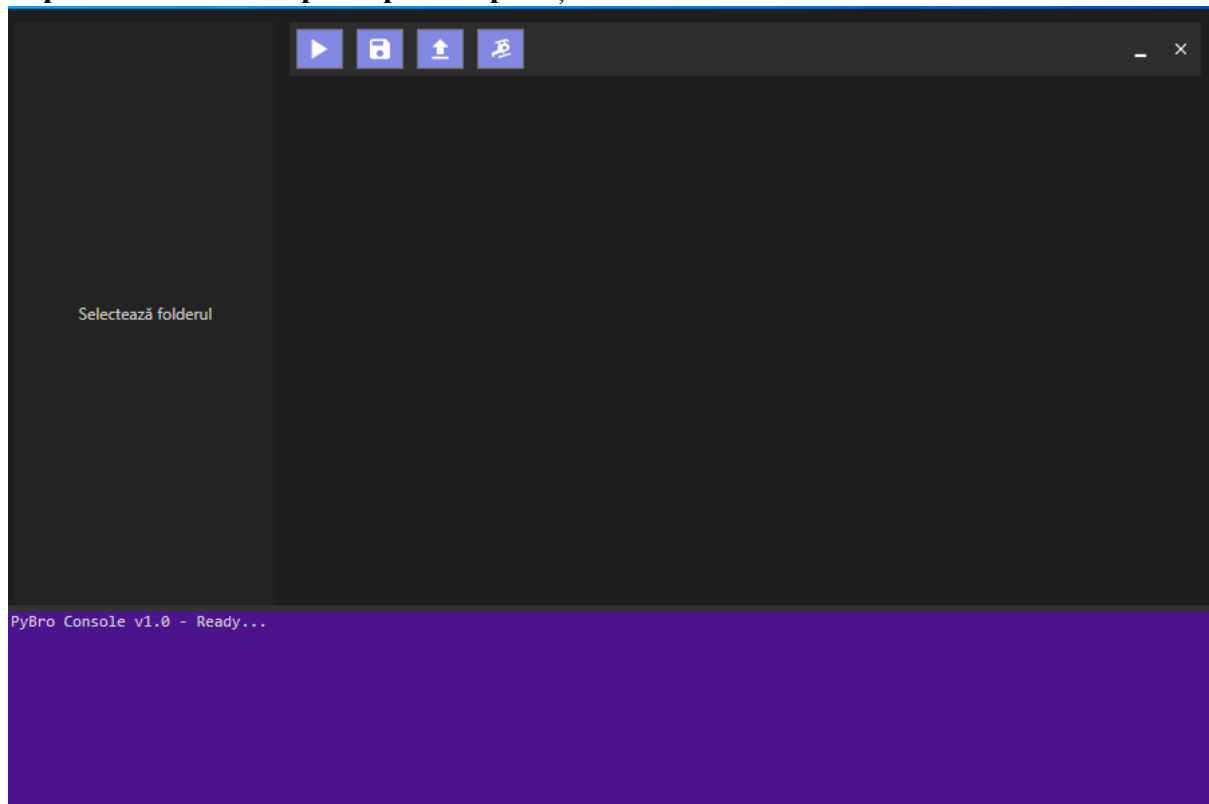
## 8. Respectarea standardelor de codare

Codul sursă al aplicației PyBro respectă convențiile de scriere C# recomandate de Microsoft (Microsoft C# Coding Conventions). Principalele convenții aplicate sunt:

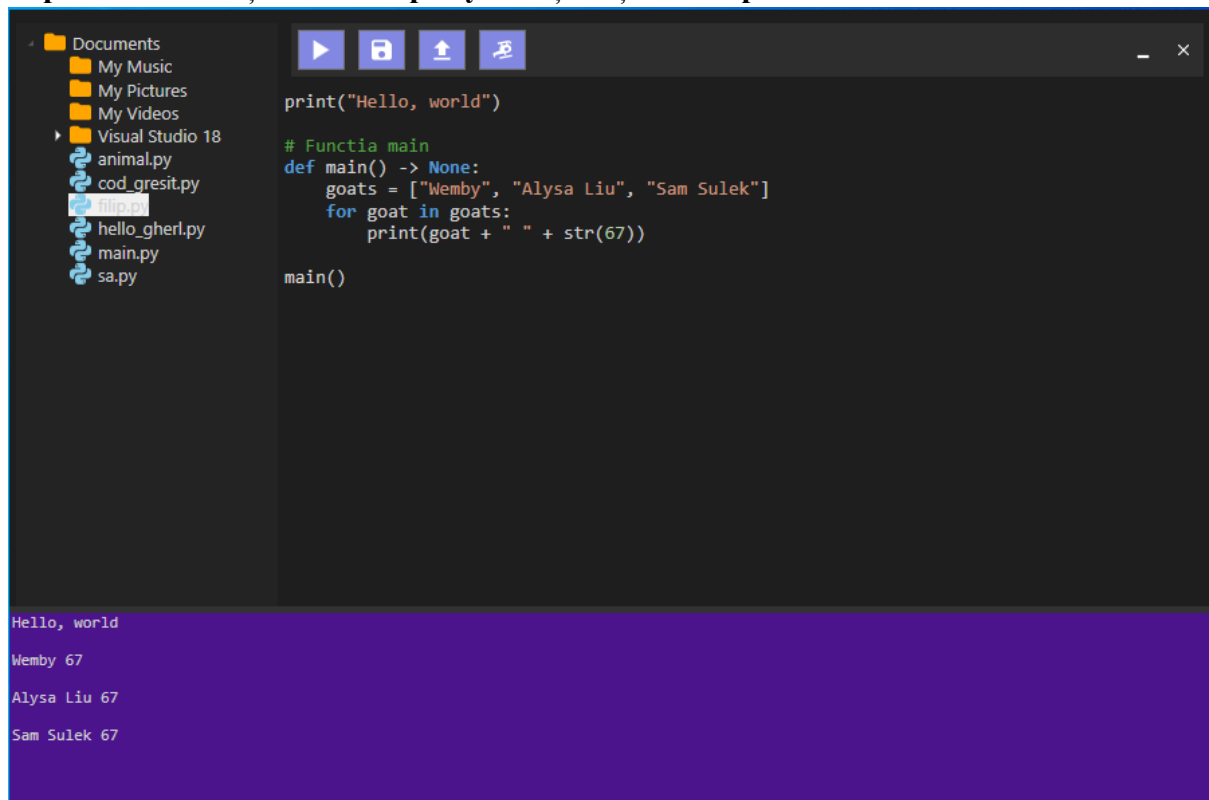
- Denumirea claselor și interfețelor în PascalCase (de ex: FileManager, IFFileManager, ModelCommandRunPythonScript)
- Denumirea câmpurilor private cu prefix underscore și camelCase (de ex: \_fileManager, \_pythonInterpreter, \_scriptEngine)
- Denumirea metodelor publice în PascalCase (de ex: CreateFile, SaveBuffer, GetFileContent, RunScript)
- Denumirea parametrilor în camelCase (de ex: path, buffer, script, directoryPath)
- Interfețele prefixate cu litera I (de ex: IFFileManager, IPythonInterpreter, ITextBuffer)
- Fiecare fișier sursă conține un antet cu informații despre autor, proiect și funcționalitate
- Fiecare clasă și metodă publică este documentată cu comentarii XML summary (<summary>)
- Validarea argumentelor în toate metodele publice cu mesaje de eroare clare

## 9. Capturi de ecran

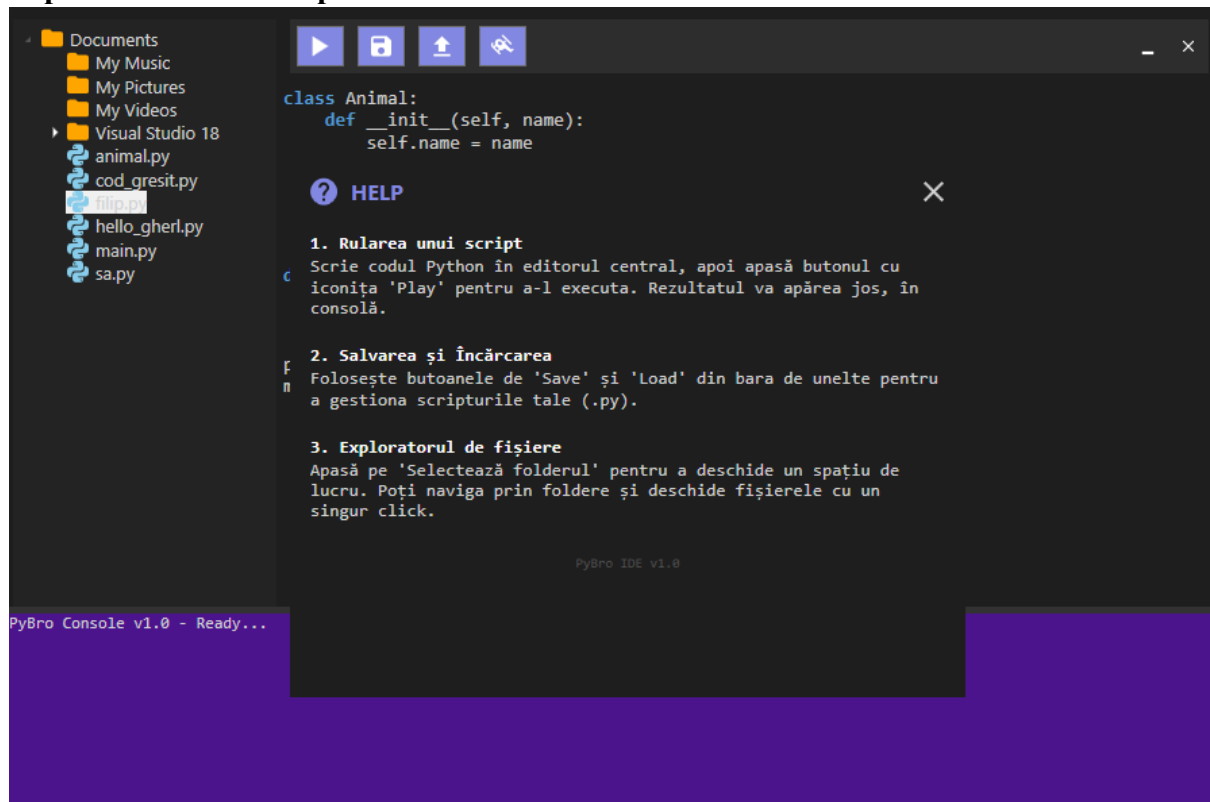
**Captură 1: Fereastra principală a aplicației**



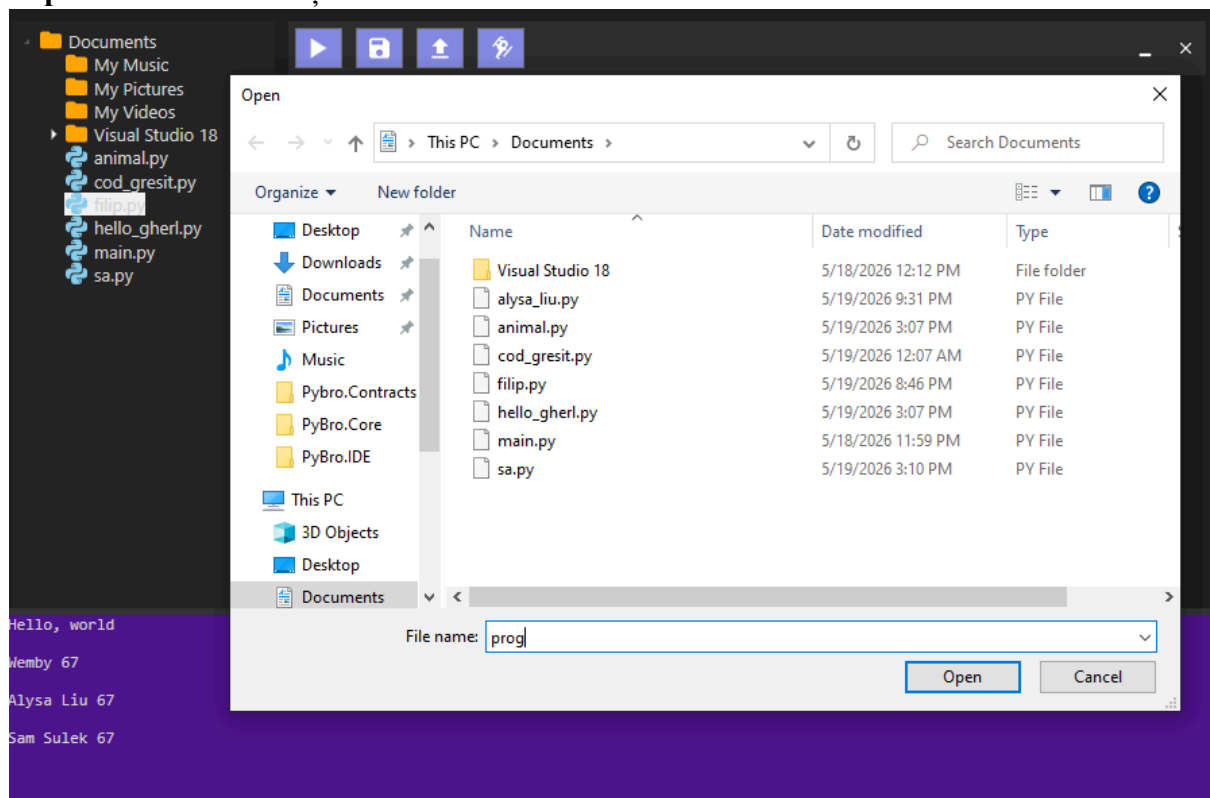
**Captură 2: Execuția unui script Python și afișarea output-ului în consolă**



### Captură 3: Meniul Help

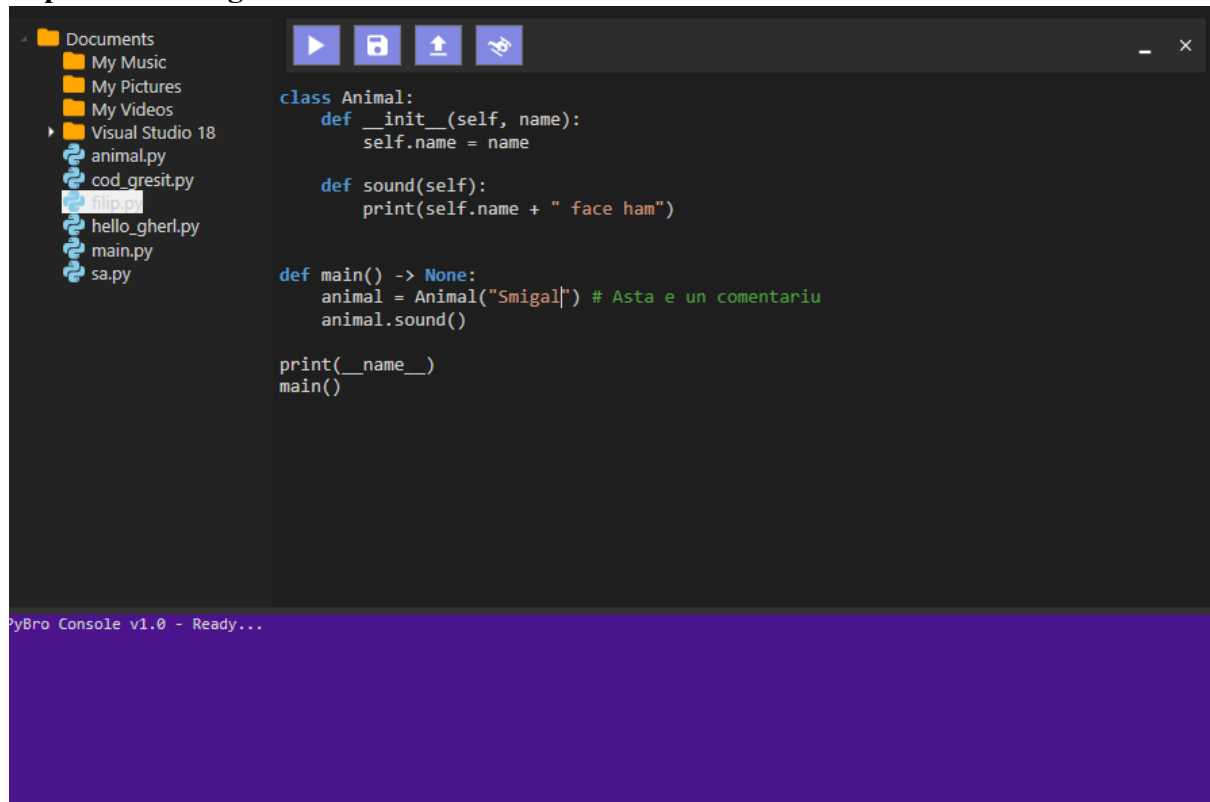


### Captură 4: Salvarea fișierelor





## Captură 5: Navigarea în arborele de directoare



## Lista contribuțiilor membrilor echipei

| Nume                         | Modul / Componentă                           | Contribuție                                                                                                                                                                                                                                                                                     |
|------------------------------|----------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Filip Andrei-Robert          | PyBro.Core,<br>PyBro.IDE,<br>Pybro.Contracts | Implementare PythonInterpreter (IronPython), FileManager, comunicare bidirecțională Model-View prin cozi de mesaje (șablon Command), auto-indentare (Python parser simplu), șablon Builder (IdeBuilder), șablon Adapter (UiAdapter), exception handling pentru PythonInterpreter și FileManager |
| Beligan Tudor                | PyBro.Ui                                     | Design UI, implementare MainWindow (WPF), TreeDir (afișare recursivă director), Syntax Highlighting (AvalonEdit), UX pentru afișarea output-ului în consolă, UX pentru creare/salvare/încărcare fișiere, Meniu Help                                                                             |
| Pînzaru<br>Alexandru-Gabriel | PyBro.Core,<br>Pybro.Contracts               | Organizarea modulelor în DLL-uri separate, comentare cod (headere + XML Documentation Comments), implementare Vibes, actualizare PythonInterpreter pentru ScriptScope                                                                                                                           |
| Modreanu Ștefan              | PyBro.Tests,<br>Documentație                 | 21 teste unitare XUnit pentru FileManager, testarea cazurilor excepționale, Document SRS, Diagrama de clase, Diagrama de activități, Diagrama de secvențe                                                                                                                                       |

## Anexă: Listing cod sursă

Sunt prezentate mai jos fragmentele semnificative din codul sursă al aplicației, comentate.

### FileManager.cs

Gestionează operațiile cu fișiere: creare, citire, scriere, ștergere.

```
public class FileManager : IFileManager
{
    /// <summary>Creates an empty file at the specified path.</summary>
    public void CreateFile(string path)
    {
        try { File.Create(path).Dispose(); }
        catch { throw new FileNotFoundException("Can not create file " + path); }
    }

    /// <summary>Saves text buffer to file; overwrites if exists.</summary>
    public void SaveBuffer(string path, string buffer)
    {
        try
        {
            var dir = GetDirectoryName(path);
            EnsureDirectoryExists(dir);
            File.WriteAllText(path, buffer);
        }
        catch { throw new FileNotFoundException("Can not write to file " + path); }
    }

    /// <summary>Reads file content and returns it as string.</summary>
    public string GetFileContent(string path)
    {
        try { return File.ReadAllText(path); }
        catch { throw new FileNotFoundException("Can not read from file " + path); }
    }

    /// <summary>Removes the file at the specified path.</summary>
    public void RemoveFile(string path)
    {
        if (!File.Exists(path))
            throw new FileNotFoundException("File " + path + " does not exist!");
        File.Delete(path);
    }
}
```

### PythonInterpreter.cs

Execută scripturi Python prin IronPython și captează stdout/stderr.

```
public class PythonInterpreter : IPythonInterpreter
{
    private readonly ScriptEngine _scriptEngine;
    private readonly ScriptScope _scriptScope;

    public PythonInterpreter()
```

```

{
    _scriptEngine = Python.CreateEngine();
    _scriptScope = _scriptEngine.CreateScope();
}

/// <summary>Runs script, returns (stdout, stderr) tuple.</summary>
public (string, string) RunScript(string script)
{
    try
    {
        using var stdout = new MemoryStream();
        using var stderr = new MemoryStream();
        _scriptEngine.Runtime.IO.SetOutput(stdout, Encoding.UTF8);
        _scriptEngine.Runtime.IO.SetErrorOutput(stderr, Encoding.UTF8);
        _scriptEngine.Execute(script, _scriptScope);
        stdout.Position = 0; stderr.Position = 0;
        return (new StreamReader(stdout).ReadToEnd(),
            new StreamReader(stderr).ReadToEnd());
    }
    catch (IronPython.Runtime.Exceptions.RuntimeException ex)
    {
        throw new Exception("IronPython Runtime Exception: " +
ex.Message);
    }
    catch
    {
        throw new PythonException("Error while trying to run the python
script!");
    }
}
}

```

### ArrayListTextBuffer.cs

Buffer de text cu suport pentru auto-indentare.

```

public class ArrayListTextBuffer : ITextBuffer
{
    private List<string> _buffer { get; set; }

    /// <summary>Calculates current indentation level and string.</summary>
    public (uint, string?) GetCurrentIndentInfo()
    {
        uint indentLevel = 0;
        string? indentString = null;
        for (int i = 0; i < _buffer.Count; ++i)
        {
            var line = _buffer[i];
            if (indentLevel >= 1 && indentString == null)
                indentString = GetIndentString(line);
            if (ShouldIncrementIndentation(line))
                indentLevel++;
        }
        return (indentLevel, indentString);
    }

    /// <summary>Applies auto-indentation to last empty line.</summary>
    public void Autoindent()

```

```

    {
        var (indentLevel, indentString) = this.GetCurrentIndentInfo();
        if (_buffer.Last() == "")
        {
            var sb = new StringBuilder();
            for (int i = 0; i < indentLevel; ++i) sb.Append(indentString);
            _buffer[_buffer.Count - 1] = sb.ToString();
        }
    }
}

```

## Controller.cs

Coordonează Model și View prin ciclul Tick, apelat la fiecare 16ms.

```

public class Controller(IModel model, IView view) : IController
{
    private readonly IModel _model = model;
    private readonly IView _view = view;

    /// <summary>Executes pending commands for both Model and
    View.</summary>
    public void Tick()
    {
        _model.ExecuteCommands();
        _view.ExecuteCommands();
    }
}

```

## MessageQueues.cs

Cozi concurente thread-safe pentru comunicarea dintre Model și View.

```

public class MessageQueues
{
    private static readonly ConcurrentQueue<IModelCommand>
        _viewToModelCommandQueue = new();
    private static readonly ConcurrentQueue<IViewCommand>
        _modelToViewCommandQueue = new();

    public static void SendModelCommand(IModelCommand cmd)
        => _viewToModelCommandQueue.Enqueue(cmd);

    public static IModelCommand? ReceiveModelCommand()
    {
        if (_viewToModelCommandQueue.TryDequeue(out var item)) return item;
        return null;
    }

    public static void SendViewCommand(IViewCommand cmd)
        => _modelToViewCommandQueue.Enqueue(cmd);

    public static IViewCommand? ReceiveViewCommand()
    {
        if (_modelToViewCommandQueue.TryDequeue(out var item)) return item;
        return null;
    }
}

```

} }